Московский Авиационный Институт

(Национальный Исследовательский Университет)

Институт №8 “Компьютерные науки и прикладная математика”

Кафедра №806 “Вычислительная математика и программирование”

Лабораторная работа №2 по курсу

«Операционные системы»

Группа: М8О-209БВ-25

Студент: Леваков Н.Д.

Преподаватель: Бахарев В.Д.

Оценка: _____

Дата: 18.10.26

Москва, 2026

Постановка задачи

Вариант 19.

Составить программу на языке Си, обрабатывающую данные в многопоточном режиме. При обработки использовать стандартные создания потоков операционной системы (Windows/Unix). Ограничение максимального количества потоков, работающих в один момент времени, должно быть задано ключом запуска вашей программы.

Также необходимо продемонстрировать количество потоков, используемое вашей программой с помощью стандартных средств операционной системы.

Дан массив координат (x, y). Пользователь вводит число кластеров. Проведите кластеризацию методом k-средних.

Общий метод и алгоритм решения

Использованные системные вызовы:

- pthread_create(); - создает новый поток. Принимает указатель на переменную для хранения ID потока, атрибуты, функцию потока и аргументы для неё.
- pthread_join(); - блокирует вызывающий поток до тех пор, пока указанный поток не завершит свое выполнение. Нужен для синхронизации.
- pthread_mutex_lock(); -захватывает мьютекс. Если мьютекс уже занят другим потоком, вызывающий поток блокируется до его осовбождения.
- pthread_mutex_unlock(); - освобождает мьютекс, позволяя другим потокам войти в критическую секцию.
- pthread_mutex_t; - тип данных для мьютекса.
- clock_gettime(); - системный вызов для замера времени. Используется с монотонным таймером CLOCK_MONOTONIC, который не зависит от изменения системного вызова.
- calloc()/malloc()/free(); - выделение и освобождение памяти в куче, calloc используется для массивов, так как обнуляет память.

В лабораторной работе была реализована кластеризация 1000000 точек на плоскости методов k-средних. Было написано две версии программы: последовательная и параллельная. Алгоритм k-средних работает следующим образом:

1) Случайным образом генерируются 1000000 точек. Первые k точек назначаются начальными центроидами кластеров.
2) Для каждой точки вычисляется евклидово расстояние до каждого центроида. Точка привязывается к ближайшему кластеру.
3) Для каждого кластера вычисляется новый центроид как среднее арифметическое координат всех его точек
4) Если максимальное смещение центроидов меньше EPSILON (0.001), то алгоритм завершается. Иначе - повтор со второго шага.

В последовательной версии один поток выполняет шаги 2-3 последовательно для всех точек и кластеров. Время работы замеряется с помощью clock_gettime().

В параллельной версии для распараллеливания были выделены два этапа:

1) Параллельное присваивание: массив делится на N диапазонов (по числу потоков). Каждый поток независимо ищет ближайший центроид для своих точек и записывает

номер кластера в массив. Мьютекс здесь не нужен, так как потоки пишут в разные ячейки памяти.

2) Параллельный подсчет сумм: каждый поток считает здесь свои локальные суммы координат X и Y, а также количество точек для своих кластеров. Затем потоки синхронизируются через мьютекс, чтобы безопасно добавить свои локальные результаты в глобальные массивы sum_x, sum_y, count.

Также происходит последовательное обновление центроидов: после того как все потоки завершили подсчет, главный поток вычисляет новые координаты центроидов и проверяет сходимость.

Результаты экспериментов:

Характеристики системы: Intel Core i5-12400F, 6 физических ядер и 12 логических ядер.

Метрики:

Ускорение: S = T_seq / T_par

Эффективность: X = S / p, где p – это количество ядер

Число потоков	Время (мс)	Ускорение (S)	Эффективность (X)
Последовательная версия	2124.18	-	-
1	1931.82	1.1	0.09
2	1313.13	1.6	0.13
4	942.39	2.25	0.19
8	432.69	4.9	0.4
12	305.47	6.95	0.58
16	428.09	4.96	0.41
128	560.59	3.79	0.32
1024	2921.48	0.73	0.06

Код программы

kmeans_seq.c

```c
#include <stdio.h>
#include <stdlib.h>
#include <math.h>
#include <time.h>

#define N_POINTS 1000000
#define MAX_ITER 100
#define EPSILON 0.001

typedef struct {
    double x, y;
    int cluster;
} Point;
```

```c
double distance(Point p, double centre_x, double centre_y) {
    return sqrt((p.x - centre_x) * (p.x - centre_x) + (p.y - centre_y) * (p.y -
centre_y));
}

int main(int argc, char *argv[]) {
    int N_CLUSTERS = 5;

    if (argc > 1) {
        N_CLUSTERS = atoi(argv[1]);
        if (N_CLUSTERS < 1)
            N_CLUSTERS = 1;
        if (N_CLUSTERS > N_POINTS)
            N_CLUSTERS = N_POINTS;
    }

    srand(time(NULL));
    Point *points = malloc(N_POINTS * sizeof(Point));

    for (int i = 0; i < N_POINTS; i++) {
        points[i].x = (double)rand() / RAND_MAX * 100.0;
        points[i].y = (double)rand() / RAND_MAX * 100.0;
        points[i].cluster = -1;
    }

    double *centre_x = malloc(N_CLUSTERS * sizeof(double));
    double *centre_y = malloc(N_CLUSTERS * sizeof(double));
    for (int k = 0; k < N_CLUSTERS; k++) {
        centre_x[k] = points[k].x;
        centre_y[k] = points[k].y;
    }

    struct timespec start, end;
    clock_gettime(CLOCK_MONOTONIC, &start);

    double *sum_x = calloc(N_CLUSTERS, sizeof(double));
    double *sum_y = calloc(N_CLUSTERS, sizeof(double));
    int *count = calloc(N_CLUSTERS, sizeof(int));

    for (int iter = 0; iter < MAX_ITER; iter++) {
        for (int i = 0; i < N_POINTS; i++) {
            double min_distance = distance(points[i], centre_x[0], centre_y[0]);
            int best = 0;
            for(int k = 1; k < N_CLUSTERS; k++) {
                double temp_dist = distance(points[i], centre_x[k], centre_y[k]);
                if (temp_dist < min_distance) {
                    min_distance = temp_dist;
                    best = k;
                }
            }
            points[i].cluster = best;
        }
```

```
        for (int k= 0; k < N_CLUSTERS; k++) {
            sum_x[k] = 0;
            sum_y[k] = 0;
            count[k] = 0;
        }

        for (int i = 0; i < N_POINTS; i++) {
            int k = points[i].cluster;
            sum_x[k] += points[i].x;
            sum_y[k] += points[i].y;
            count[k]++;
        }

        double max_shift = 0.0;
        for (int k = 0; k < N_CLUSTERS; k++) {
            if (count[k] > 0) {
                double new_centre_x = sum_x[k] / count[k];
                double new_centre_y = sum_y[k] / count[k];
                double shift = sqrt((new_centre_x - centre_x[k]) * (new_centre_x -
centre_x[k]) +
                (new_centre_y - centre_y[k]) * (new_centre_y - centre_y[k]));
                if (shift > max_shift)
                    max_shift = shift;
                centre_x[k] = new_centre_x;
                centre_y[k] = new_centre_y;
            }
        }

        if (max_shift < EPSILON) {
            printf("Сошлось на итерации: %d\n", iter + 1);
            break;
        }
    }

    free(sum_x);
    free(sum_y);
    free(count);

    clock_gettime(CLOCK_MONOTONIC, &end);
    double time_ms = (end.tv_sec - start.tv_sec) * 1000.0 + (end.tv_nsec - start.tv_nsec)
/ 1e6;
    printf("Время: %.2f мс\n", time_ms);

    int *cluster_size = calloc(N_CLUSTERS, sizeof(int));
    for (int i = 0; i < N_POINTS; i++)
        cluster_size[points[i].cluster]++;

    printf("Размеры кластеров:\n");
    for (int k = 0; k < N_CLUSTERS; k++)
        printf("Кластер %d: %d точек\n", k, cluster_size[k]);

    free(cluster_size);
    free(points);
    free(centre_x);
```

```
    free(centre_y);
    return 0;
}
```

kmeans_par.c

```
#include <stdio.h>
#include <stdlib.h>
#include <math.h>
#include <time.h>
#include <pthread.h>
#include <string.h>

#define N_POINTS 1000000
#define MAX_ITER 100
#define EPSILON 0.001

typedef struct {
    double x, y;
    int cluster;
} Point;

typedef struct {
    Point *points;
    int start, end;
    double *centre_x;
    double *centre_y;
    int N_CLUSTERS;
    double *sum_x;
    double *sum_y;
    int *count;
    pthread_mutex_t *mutex;
} ThreadArgs;

double distance(Point p, double centre_x, double centre_y) {
    return sqrt((p.x - centre_x) * (p.x - centre_x) + (p.y - centre_y) * (p.y -
centre_y));
}

void *assign_task(void * _args) {
    ThreadArgs *args = (ThreadArgs *)_args;

    for (int i = args->start; i < args->end; i++) {
        double min_distance = distance(args->points[i], args->centre_x[0], args-
>centre_y[0]);
        int best = 0;

        for (int k = 1; k < args->N_CLUSTERS; k++) {
            double temp_dist = distance(args->points[i], args->centre_x[k], args-
>centre_y[k]);
            if (temp_dist < min_distance) {
                min_distance = temp_dist;
```

```
                best = k;
            }
        }
        args->points[i].cluster = best;
    }
    return NULL;
}

void *update_task(void *_args) {
    ThreadArgs *args = (ThreadArgs *)_args;
    double *local_sum_x = calloc(args->N_CLUSTERS, sizeof(double));
    double *local_sum_y = calloc(args->N_CLUSTERS, sizeof(double));
    int *local_count = calloc(args->N_CLUSTERS, sizeof(int));

    for (int i = args->start; i < args->end; i++) {
        int k = args->points[i].cluster;
        local_sum_x[k] += args->points[i].x;
        local_sum_y[k] += args->points[i].y;
        local_count[k]++;
    }

    pthread_mutex_lock(args->mutex);
    for (int k = 0; k < args->N_CLUSTERS; k++) {
        args->sum_x[k] += local_sum_x[k];
        args->sum_y[k] += local_sum_y[k];
        args->count[k] += local_count[k];
    }
    pthread_mutex_unlock(args->mutex);

    free(local_sum_x);
    free(local_sum_y);
    free(local_count);
    return NULL;
}

int main(int argc, char *argv[]) {
    int N_THREADS = 4;
    int N_CLUSTERS = 5;

    if (argc > 1)
        N_THREADS = atoi(argv[1]);

    if (argc > 2)
        N_CLUSTERS = atoi(argv[2]);

    if (N_THREADS < 1)
        N_THREADS = 1;

    if (N_CLUSTERS < 1)
        N_CLUSTERS = 1;

    if (N_CLUSTERS > N_POINTS)
        N_CLUSTERS = N_POINTS;
```

```
    srand(time(NULL));
    Point *points = malloc(N_POINTS * sizeof(Point));

    for (int i = 0; i < N_POINTS; i++) {
        points[i].x = (double)rand() / RAND_MAX * 100.0;
        points[i].y = (double)rand() / RAND_MAX * 100.0;
        points[i].cluster = -1;
    }

    double *sum_x = calloc(N_CLUSTERS, sizeof(double));
    double *sum_y = calloc(N_CLUSTERS, sizeof(double));
    int *count = calloc(N_CLUSTERS, sizeof(int));

    double *centre_x = malloc(N_CLUSTERS * sizeof(double));
    double *centre_y = malloc(N_CLUSTERS * sizeof(double));
    for (int k = 0; k < N_CLUSTERS; k++) {
    centre_x[k] = points[k].x;
    centre_y[k] = points[k].y;
    }

    pthread_t *threads = malloc(N_THREADS * sizeof(pthread_t));
    ThreadArgs *args = malloc(N_THREADS * sizeof(ThreadArgs));
    pthread_mutex_t mutex = PTHREAD_MUTEX_INITIALIZER;

    int chunk = N_POINTS / N_THREADS;
    if (chunk == 0) { chunk = 1; }

    for (int t = 0; t < N_THREADS; t++) {
        args[t].points = points;
        args[t].centre_x = centre_x;
        args[t].centre_y = centre_y;
        args[t].N_CLUSTERS = N_CLUSTERS;
        args[t].sum_x = sum_x;
        args[t].sum_y = sum_y;
        args[t].count = count;
        args[t].mutex = &mutex;

        args[t].start = t * chunk;
        args[t].end = ( t == N_THREADS - 1 ) ? N_POINTS : (t + 1) * chunk;

        if (args[t].start > N_POINTS)
            args[t].start = N_POINTS;

        if (args[t].end > N_POINTS)
            args[t].end = N_POINTS;
    }

    struct timespec start, end;
    clock_gettime(CLOCK_MONOTONIC, &start);

    for (int iter = 0; iter < MAX_ITER; iter++) {
        for (int t = 0; t < N_THREADS; t++)
            pthread_create(&threads[t], NULL, assign_task, &args[t]);
```

```
        for (int t = 0; t < N_THREADS; t++)
            pthread_join(threads[t], NULL);

        for (int k = 0; k < N_CLUSTERS; k++) {
            sum_x[k] = 0;
            sum_y[k] = 0;
            count[k] = 0;
        }

        for (int t = 0; t < N_THREADS; t++)
            pthread_create(&threads[t], NULL, update_task, &args[t]);

        for (int t = 0; t < N_THREADS; t++)
            pthread_join(threads[t], NULL);

        double max_shift = 0.0;
        for (int k = 0; k < N_CLUSTERS; k++) {
            if (count[k] > 0) {
                double new_centre_x = sum_x[k] / count[k];
                double new_centre_y = sum_y[k] / count[k];
                double shift = sqrt((new_centre_x - centre_x[k]) * (new_centre_x -
centre_x[k]) +
                (new_centre_y - centre_y[k]) * (new_centre_y - centre_y[k]));

                if (shift > max_shift)
                    max_shift = shift;

                centre_x[k] = new_centre_x;
                centre_y[k] = new_centre_y;
            }
        }

        if (max_shift < EPSILON) {
            printf("Сошлось на итерации: %d\n", iter + 1);
            break;
        }
    }

    clock_gettime(CLOCK_MONOTONIC, &end);
    double time_ms = (end.tv_sec - start.tv_sec) * 1000.0 + (end.tv_nsec - start.tv_nsec)
/ 1e6;
    printf("Параллельная версия. Потоков - %d, кластеров - %d. Время работы: %.2f мс\n",
N_THREADS, N_CLUSTERS, time_ms);

    int *cluster_size = calloc(N_CLUSTERS, sizeof(int));
    for (int i = 0; i < N_POINTS; i++)
        cluster_size[points[i].cluster]++;

    printf("Размеры кластеров:\n");
    for (int k = 0; k < N_CLUSTERS; k++)
        printf("Кластер %d: %d точек\n", k, cluster_size[k]);

    free(cluster_size);
    free(points);
```

```
    free(centre_x);
    free(centre_y);
    free(sum_x);
    free(sum_y);
    free(count);
    free(threads);
    free(args);

    return 0;
}
```

Протокол работы программы

Тестирование:

>>> gcc -o kmeans_seq kmeans_seq.c -lm

>>> gcc -o kmeans_par kmeans_par.c -lm

>>> ./kmeans_seq

<<< Сошлось на итерации: 45

Время: 2124.18 мс

Размеры кластеров:

Кластер 0: 171880 точек

Кластер 1: 171863 точек

Кластер 2: 237815 точек

Кластер 3: 237637 точек

Кластер 4: 180805 точек

>>> ./kmeans_par 1

<<< Сошлось на итерации: 39

Параллельная версия. Потоков - 1, кластеров - 5. Время работы: 1931.82 мс

Размеры кластеров:

Кластер 0: 211530 точек

Кластер 1: 210040 точек

Кластер 2: 211323 точек

Кластер 3: 155920 точек

Кластер 4: 211187 точек

>>> ./kmeans_par 2

<<< Сошлось на итерации: 47

Параллельная версия. Потоков - 2, кластеров - 5. Время работы: 1313.13 мс

Размеры кластеров:

Кластер 0: 171146 точек

Кластер 1: 237815 точек

Кластер 2: 181006 точек

Кластер 3: 172175 точек

Кластер 4: 237858 точек

>>> ./kmeans_par 4

<<< Сошлось на итерации: 54

Параллельная версия. Потоков - 4, кластеров - 5. Время работы: 942.39 мс

Размеры кластеров:

Кластер 0: 172538 точек

Кластер 1: 238098 точек

Кластер 2: 170832 точек

Кластер 3: 180914 точек

Кластер 4: 237618 точек

>>> ./kmeans_par 8

<<< Сошлось на итерации: 43

Параллельная версия. Потоков - 8, кластеров - 5. Время работы: 432.69 мс

Размеры кластеров:

Кластер 0: 237286 точек

Кластер 1: 238799 точек

Кластер 2: 180661 точек

Кластер 3: 172058 точек

Кластер 4: 171196 точек

>>> ./kmeans_par 12

<<< Сошлось на итерации: 30

Параллельная версия. Потоков - 12, кластеров - 5. Время работы: 305.47 мс

Размеры кластеров:

Кластер 0: 172249 точек

Кластер 1: 170826 точек

Кластер 2: 238178 точек

Кластер 3: 180865 точек

Кластер 4: 237882 точек

>>> ./kmeans_par 16

<<< Сошлось на итерации: 42

Параллельная версия. Потоков - 16, кластеров - 5. Время работы: 428.09 мс

Размеры кластеров:

Кластер 0: 238682 точек

Кластер 1: 237051 точек

Кластер 2: 181563 точек

Кластер 3: 171691 точек

Кластер 4: 171013 точек

>>> ./kmeans_par 128

<<< Сошлось на итерации: 44

Параллельная версия. Потоков - 128, кластеров - 5. Время работы: 560.59 мс

Размеры кластеров:

Кластер 0: 238954 точек

Кластер 1: 171354 точек

Кластер 2: 181049 точек

Кластер 3: 171225 точек

Кластер 4: 237418 точек

>>> ./kmeans_par 1024

<<< Сошлось на итерации: 39

Параллельная версия. Потоков - 1024, кластеров - 5. Время работы: 2921.48 мс

Размеры кластеров:

Кластер 0: 172225 точек

Кластер 1: 237016 точек

Кластер 2: 172364 точек

Кластер 3: 180799 точек

Кластер 4: 237596 точек

Вывод

В ходе работы было установлено, что максимальное ускорение (6.95) достигается при использовании 12 потоков, что соответствует количеству логических ядер процессора, при этом эффективность составляет 0.58. При дальнейшем увеличении числа потоков (16, 128, 1024) ускорение падает из-за накладных расходов на переключение контеста и псевдопараллелизма, а при 1024 потоках программа работает даже медленнее последовательной версии. Также было подтверждено ограничение ускорения законом Амдала: наличие последовательных участков кода не позволяет достичь идеального линейного ускорения. Таким образом, для эффективного распараллеливания количество потоков должно соответствовать числу логических ядер, а не превышать его.